

Hotel Booking Platform

Interview Preparation Guide

A full-stack Django 6.0 web application

Built with Python, Django ORM, SQLite/PostgreSQL, and Pillow

Use this guide to confidently present and discuss your project

Table of Contents

1. Project Overview & Elevator Pitch
2. Architecture & Design Decisions
3. Key Features & Walkthrough
4. Technical Highlights to Emphasize
5. Common Interview Questions & Answers
6. Code You Should Be Ready to Explain
7. Challenges & How You Solved Them
8. What You Learned & Future Improvements
9. Quick Reference Cheat Sheet

1. Project Overview & Elevator Pitch

Elevator Pitch (30 seconds)

"This is a Hotel Booking Platform built with Django 6.0. It is a full-stack web application that allows hotel owners to list and manage their properties, customers to browse hotels and rooms, make bookings, process payments, and leave reviews. It uses Django's ORM for all database operations, a custom user model with role-based authorization, and follows the MVT pattern with class-based views. The project has 7 Django apps, 179 automated tests, and is production-ready with deployment tooling for PythonAnywhere."

Project Overview (1 minute)

The application serves two main user roles: hotel owners and customers. Owners can create and manage hotels and rooms. Customers can browse listings, filter by city/name/price, book rooms, make payments, and leave reviews. The project uses Django 6.0 with SQLite for development and PostgreSQL for production. Static files are served via WhiteNoise, and media files can use Cloudinary for production. The project includes a full test suite with 179 tests covering models, forms, views, and URL routing.

Why This Project Matters

- Demonstrates understanding of Django's full stack: models, views, templates, URLs, forms
- Shows production awareness: deployment config, environment variables, separate dev/prod settings
- Proves testing discipline: 179 tests across all 7 apps
- Exhibits security awareness: role-based auth, user-scoped data access, CSRF protection
- Covers real-world relational data: users, hotels, rooms, bookings, payments, reviews

2. Architecture & Design Decisions

Seven Django Apps — Separation of Concerns

accounts	Custom user auth, signup, profile, password management (email-based login)
pages	Home page with featured hotels and hero section
hotels	Hotel CRUD — owner-gated create/update/delete with public listing/detail
rooms	Room CRUD per hotel — owner-gated with public browse/filter
bookings	Booking CRUD per user — auto-computes total price
payments	Payment per booking (OneToOne) — auto-sets amount from booking
reviews	Reviews per hotel — one per user per hotel with UniqueConstraint

Data Model Relationships

CustomUser (1) <-> (1) Profile | User (1) -> (M) Booking | Hotel (1) -> (M) Room

Hotel (M) <- (1) Owner (User) | Booking (1) -> (1) Payment | Hotel (M) <- (1) User (Review)

Booking (M) <- (1) Room | Review (M) <- (1) User, Hotel

Key Design Decisions

- Custom User Model: Email-based login (USERNAME_FIELD = 'email'), keeps username in REQUIRED_FIELDS
- Role-based authorization: 'customer' vs 'owner' role field; owner-gated via UserPassesTestMixin
- Profile auto-creation: post_save signal on CustomUser creation
- Class-based views throughout: DRY, consistent pattern, built-in pagination/form handling
- Django 6.0 compatibility: uses updated django.conf.urls.static import, handles handle_no_permission changes

Tech Stack

- Backend: Python 3.13, Django 6.0, Django ORM, Whitenoise
- Database: SQLite (dev) / PostgreSQL (prod) via dj-database-url
- Image Processing: Pillow
- Media Storage: FileSystemStorage (dev) / Cloudinary (prod)
- Testing: Django TestCase (179 tests)
- Deployment: PythonAnywhere WSGI with environment-based configuration

3. Key Features & Walkthrough

How to Demo the Application

Feature 1 — Authentication & User Roles

Users sign up with email, username, password, and select a role (customer or owner). Login is email-based. Owners can manage hotels/rooms. Customers browse, book, and review. Password reset uses console email backend for development.

```
# CustomUser model
class CustomUser(AbstractUser):
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=10,
        choices=[('customer', 'Customer'), ('owner', 'Owner')],
        default="customer")
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username']
```

Feature 2 — Hotel & Room Management (Owner)

Owners create hotels with name, description, address, city, country, and image. Each hotel can have multiple rooms with types (single/double/deluxe/suite/family), pricing, and availability tracking. Owner-only views use `UserPassesTestMixin`.

Feature 3 — Search, Filter & Sort (Public)

`HotelListView` supports search by name, filter by city, and sort by name or newest. `RoomListView` supports search by room number or hotel name, filter by room type and hotel. Both use Django ORM querysets with pagination (9 per page for hotels, 12 for rooms).

Feature 4 — Booking with Price Computation

Customers select a room, choose dates and guest count. The system computes `total_price = room.price_per_night * (check_out - check_in)`. Bookings have status tracking (pending/confirmed/cancelled/completed). Users see only their own bookings.

```
# Auto-compute total price on booking
def form_valid(self, form):
    days = (form.instance.check_out - form.instance.check_in).days
    form.instance.total_price = form.instance.room.price_per_night * days
    return super().form_valid(form)
```

Feature 5 — Payments (One-to-One)

Each booking can have exactly one payment. Payment view auto-sets amount from the booking's `total_price`, validates booking ownership, and prevents duplicate payments. Supports multiple methods: card, PayPal, bank transfer, cash.

Feature 6 — Reviews with Uniqueness Constraint

Authenticated users can leave one review per hotel (enforced by database-level `UniqueConstraint`). Ratings are 1-5. Reviews display inline on the hotel detail page. Duplicate review attempts redirect to the edit view.

```
# UniqueConstraint for one review per user per hotel
class Meta:
```

```
ordering = ['-created_at']
constraints = [
    models.UniqueConstraint(
        fields=['user', 'hotel'],
        name='unique_user_hotel_review'
    )
]
```

4. Technical Highlights to Emphasize

PRO TIP

Mention these points naturally during your walkthrough. They show depth beyond basic CRUD.

Django 6.0 Awareness

You're using the latest Django. You know about the updated `django.conf.urls.static` import, the `handle_no_permission` change in `AccessMixin`, and `ImageField` behavior returning `ImageFieldFile` when empty.

Production-Ready Configuration

Environment variables for `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, `DATABASE_URL`, and `Cloudinary`. `WhiteNoise` for static files. Separate dev/prod media storage. `PythonAnywhere` WSGI script.

Role-Based Authorization

Two-tier access: customer vs owner. Owner-gated CRUD via `UserPassesTestMixin` + `LoginRequiredMixin`. User-scoped data access (users see only their bookings/payments). Django 6.0 `handle_no_permission` override.

Comprehensive Test Suite

179 tests across 7 apps. Tests cover models, forms, views, URLs, permissions, edge cases. Run with: `python manage.py test` (parallel mode available).

Database Constraints

PostgreSQL-ready via `dj-database-url`. `UniqueConstraint` for one-review-per-user-per-hotel. `ForeignKey` relationships with proper `related_name`. Null-safe image handling.

Clean Template Architecture

Project-level `base.html` with inlined CSS and CSS variables. App-level templates per concerns. Forms use consistent form-control widget class. Mobile-responsive nav with hamburger menu.

Class-Based Views Throughout

Consistent use of generic views (`ListView`, `DetailView`, `CreateView`, `UpdateView`, `DeleteView`). Built-in pagination, form validation, permission checks. DRY pattern across all apps.

5. Common Interview Questions & Answers

Q: Why Django for this project?

Django's ORM, built-in admin, authentication system, and class-based views made it ideal. It provides batteries-included security (CSRF, XSS protection) and scales well. The MVT pattern maps naturally to web applications with database relationships.

Q: How did you handle user authentication?

I used a custom user model extending `AbstractUser` with email as the login field. Users have roles (customer/owner). Profiles are auto-created via a `post_save` signal. Password management uses Django's built-in views for reset and change.

Q: How does authorization work?

Two levels: role-based (owner vs customer) using `UserPassesTestMixin`, and user-scoped (users only see their own bookings/payments) using manual checks. For owner-gated views, I combine `LoginRequiredMixin` with `UserPassesTestMixin` and override `handle_no_permission` to redirect with a message instead of raising `PermissionDenied`.

Q: How did you structure the database?

Six models across seven apps. CustomUser -> Profile (one-to-one), Hotel -> Room (one-to-many), User -> Booking (one-to-many), Room -> Booking (one-to-many), Booking -> Payment (one-to-one), and User+Hotel -> Review (unique constraint). Using Django ORM with proper foreign keys and `related_names` for efficient querying.

Q: How did you test the application?

I wrote 179 tests using Django's `TestCase`. Each app has dedicated test files covering models (creation, string representation, defaults, constraints), forms (validation, widgets), views (URL resolution, templates, permissions, context data), and edge cases (404s, duplicate entries, unauthenticated access). I can run them with `python manage.py test`.

Q: What challenges did you face?

One challenge was Django 6.0's change to `handle_no_permission` in `AccessMixin` which now raises `PermissionDenied` for authenticated users who fail `test_func()`. I had to override it with proper `is_authenticated` checking. Another was `ImageField` returning `ImageFieldFile` wrapper instead of `None` when empty, which required adjusting test assertions.

Q: How would you extend this project?

I would add real payment gateway integration (Stripe), email notifications via SendGrid, availability validation during booking (checking overlapping dates), an API layer with DRF, and a real-time notification system using Django Channels or WebSockets.

Q: How is this production-ready?

The settings use environment variables for all sensitive config. `WhiteNoise` serves static files. PostgreSQL can replace SQLite via `DATABASE_URL`. Media can use Cloudinary. The project has a PythonAnywhere WSGI entry point. Security settings like `SECURE_HSTS` and `CSRF_COOKIE_SECURE` are toggled based on `DEBUG` mode.

Q: Walk me through a booking request lifecycle.

A user selects a room and submits the booking form. BookingCreateView validates the form, computes `total_price = price_per_night * days` in `form_valid()`, and saves the booking. The user then creates a payment via PaymentCreateView, which auto-sets amount from `booking.total_price` and validates ownership. Both views use LoginRequiredMixin. The booking shows up in their booking list, ordered by most recent.

Q: How did you manage the git workflow?

I used a standard git workflow with meaningful commits. The project is version-controlled locally. I made atomic commits for features like 'Add hotel model and CRUD views' or 'Add booking with price computation'. The `.gitignore` excludes `pyc` files, media uploads, virtual environment, and the SQLite database.

Q: What would you improve with more time?

I'd add availability validation to prevent double-booking, integrate a real payment gateway, add email notifications for booking confirmations, implement an admin dashboard with analytics, write more comprehensive tests for edge cases, add Django REST Framework for mobile app support, and improve the UI with a frontend framework like HTMX.

6. Code You Should Be Ready to Explain

Be prepared to walk through these code snippets. Read them aloud and explain each line.

1. Custom User Model (accounts/models.py)

```
class CustomUser(AbstractUser):
    email = models.EmailField(unique=True)
    phone_number = models.CharField(max_length=15, blank=True)
    role = models.CharField(
        max_length=10,
        choices=[('customer', 'Customer'), ('owner', 'Owner')],
        default='customer'
    )
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username']

    def __str__(self):
        return self.email
```

Why: Email-based login is more practical. Role field enables authorization. Username kept for Django compatibility.

2. Owner-Gated View (hotels/views.py)

```
class HotelCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    model = Hotel
    form_class = HotelForm
    template_name = 'hotels/hotel_form.html'
    success_url = reverse_lazy('hotels:hotel_list')

    def test_func(self):
        return self.request.user.role == 'owner'

    def handle_no_permission(self):
        if not self.request.user.is_authenticated:
            return redirect('login')
        messages.error(self.request, 'Only owners can create hotels.')
        return redirect('home')

    def form_valid(self, form):
        form.instance.owner = self.request.user
        return super().form_valid(form)
```

Why: Shows understanding of Django's mixin-based permission system. handle_no_permission is a Django 6.0 nuance.

3. Booking with Price Auto-Compute (bookings/views.py)

```
class BookingCreateView(LoginRequiredMixin, CreateView):
    model = Booking
    form_class = BookingForm
    template_name = 'bookings/booking_form.html'
```

```

success_url = reverse_lazy('bookings:booking_list')

def get_initial(self):
    initial = super().get_initial()
    room_id = self.request.GET.get('room')
    if room_id:
        initial['room'] = room_id
    return initial

def form_valid(self, form):
    form.instance.user = self.request.user
    days = (form.instance.check_out - form.instance.check_in).days
    form.instance.total_price = form.instance.room.price_per_night * days
    return super().form_valid(form)

```

Why: Shows business logic integration (price computation), pre-selection from URL params, and user assignment.

4. UniqueConstraint for Reviews (reviews/models.py)

```

class Review(models.Model):
    user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='reviews')
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE, related_name='reviews')
    rating = models.PositiveSmallIntegerField(choices=[(i, i) for i in range(1, 6)])
    comment = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']
        constraints = [
            models.UniqueConstraint(
                fields=['user', 'hotel'],
                name='unique_user_hotel_review'
            )
        ]

```

Why: Database-level constraint ensures data integrity. Shows understanding of Meta options and constraints.

7. Challenges & How You Solved Them

Interviewers love hearing about real problems you solved. Be ready with these:

Django 6.0 handle_no_permission Change

In Django 6.0, `AccessMixin` raises `PermissionDenied` for authenticated users who fail `test_func()`, even without `raise_exception=True`. This caused owners-only views to crash instead of redirect. I overrode `handle_no_permission` to check `is_authenticated` first, sending customers to login if unauthenticated or showing a message if authenticated.

ImageField Returning Wrapper Instead of None

When testing, I discovered that an empty `ImageField` returns an `ImageFieldFile` wrapper, not `None`. This broke assertions. I changed tests from `assertIsNone` to `assertFalse` to handle this Django 6.0 behavior.

One Review Per User Per Hotel

I needed to ensure users can only leave one review per hotel. I implemented this with a database-level `UniqueConstraint` and also handled it at the view level by checking for existing reviews and redirecting to edit.

Preventing Duplicate Payments

Each booking should have only one payment. I added a check in `PaymentCreateView` that redirects if a payment already exists for the booking, preventing duplicate payment entries.

Profile Null Safety

The `profile_update` view needed to handle cases where a profile might not exist (edge case from database manipulation). I added a try-except for `Profile.DoesNotExist` that creates the profile on the fly if missing.

8. What You Learned & Future Improvements

Skills Demonstrated

- Full-stack Django development with MVT architecture
- Database modeling with relational constraints and custom user models
- Role-based authorization and user-scoped data access
- Class-based views with mixins for DRY permission handling
- Django form handling, validation, and widget customization
- Writing comprehensive tests (179 tests across 7 apps)
- Production deployment considerations (environment variables, static files, WSGI)
- Git version control with atomic commits

Future Improvements (Talk About These Ambitions)

- Real payment gateway integration (Stripe/PayPal)
- Availability validation — check overlapping booking dates
- Email notifications via SendGrid or Amazon SES
- REST API with Django REST Framework for mobile/web clients
- Docker containerization for consistent environments
- CI/CD pipeline with GitHub Actions (run tests, lint, deploy)
- Admin dashboard with booking analytics and charts
- Wishlist/favorites feature for users to save hotels
- HTMX integration for dynamic updates without full page reloads
- Internationalization (i18n) for multi-language support

9. Quick Reference Cheat Sheet

Memorize these numbers and commands for quick recall during the interview.

Key Numbers

Languages/Frameworks:	Python 3.13, Django 6.0, JavaScript (vanilla)
Django Apps:	7 (accounts, pages, hotels, rooms, bookings, payments, reviews)
Database Models:	6 (CustomUser, Profile, Hotel, Room, Booking, Payment, Review)
Tests:	179 across 7 test files
Dependencies:	11 (Django, Pillow, Whitenoise, dj-database-url, psycopg2, Cloudinary, etc.)
Templates:	19 (1 base + 18 app/project templates)
User Roles:	2 (customer, owner)

Key Commands

Run server:	<code>python manage.py runserver</code>
All tests:	<code>python manage.py test</code>
Parallel tests:	<code>python manage.py test --parallel</code>
Single app test:	<code>python manage.py test hotels</code>
Single test class:	<code>python manage.py test bookings.tests.BookingCreateViewTests</code>
Migrations:	<code>python manage.py makemigrations / migrate</code>
Superuser:	<code>python manage.py createsuperuser</code>
Activate venv:	<code>.venv\Scripts\Activate.ps1</code>

Framing Your Answers — STAR Method

When answering behavioral questions, use the STAR method:

- Situation: 'We needed a hotel booking platform with role-based access...'
- Task: 'I was responsible for designing the database and implementing the booking flow...'
- Action: 'I created 7 Django apps, each with specific responsibilities, and wrote 179 tests...'
- Result: 'The project is fully functional with role-based access, search/filter, automated pricing, and a production-ready deployment setup.'

PRO TIP

Confidence matters more than perfection. If you don't know something, say "I haven't encountered that yet, but here's how I would approach it..."